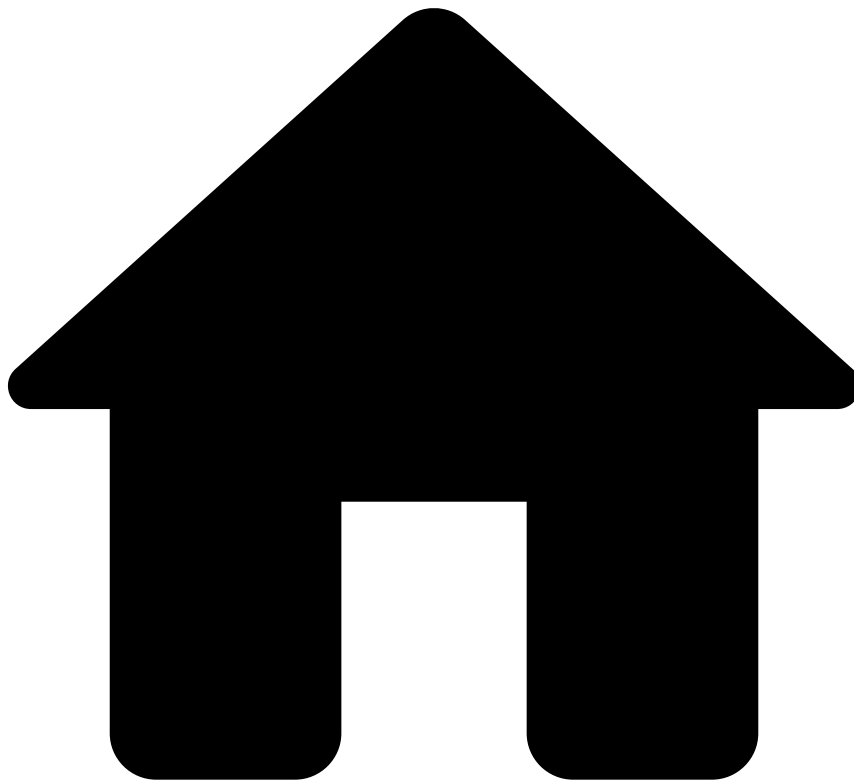


•



- Project Organization
- [Monitoring](#)
- Pulse Probes

On this page

Pulse Probes

Was this page helpful? 

Pulse probes provide a means to expose Pulse internal instrumentation data, allowing it to be stored in a central location for subsequent analysis.

Probing Components

The Pulse probing architecture is composed of two components:

- **Probes** that periodically measure some relevant metrics (for example, processing latency) and broadcast their values to a **metric server** that may be listening to them. Probing is performed either as a response to a pulse internal event (e.g. a fraud event passing by a fraud rules service workflow element) or on a periodic basis (e.g.

get sample of X every Y seconds). Each probe has an associated **metric schema**, that contains the fields that correspond to the metric. Probes are normally tightly coupled to Pulse components.

- **Metric Server**, that are subscribers of the metric information published by the probes and that store received metric information in CSV files. Metric servers may register/unregister themselves on the existing probes.

Note that the term "metric" in this context refers only to whatever is measured by probes, and should not be confused with PQL metrics.

Metric servers communicate with probes for registering/unregistering themselves and for probe discovery using a topic named `metrics_control_topic` ; metric events are broadcast to metric servers through direct TCP connections.

This architecture is extensible, allowing you to create your own probes for any pulse component you develop (e.g., custom workflow services).

Running a Metrics Server🔗

A metrics server deployment is composed of a single standalone jar file (**metrics-server-<version>.jar**) and a properties file with its configuration, named **metrics.properties** by default. A sample configuration file is included. In order to launch a metric server just run:

```
java -jar metrics-server-<version>.jar [properties_file]
```

Metrics servers can run in two modes:

- **Interactive (REPL enabled)**, where a shell is provided for registering/unregistering the server on probes in an interactive way.
- **Non-interactive (REPL disabled)**, where the server will only handle auto-started probes (see *metrics.autostartprobes* [below](#)). When used this way, the metric server must be started before the Pulse server (or, more exactly, before the probes that are to be listened for start).

The server run mode is specified in the *metrics.replenable* property of the configuration file (see [below](#)).

The *properties_file* argument can be used in case you need multiple configurations on the same host. If not specified, the default *metrics.replenable* properties will be used.

Interactive Commands🔗

The commands available in interactive mode are the following:

- `l[ist] [pattern1[,pattern2]]` - shows the available probes and their status. Absence of a pattern means all probes.
- `r[eport] pattern1[,pattern2]` - start listening on the probes specified in pattern.
- `s[ilence] pattern1[,pattern2]` - stop listening on the probes specified in pattern.

Probe pattern names support the wildcard characters `*` and `?`. The `*` wildcard matches a sequence of zero or more characters and the `?` matches exactly one character.

The list of existing commands can be displayed on the shell just by entering an empty line.



< 15.1.10

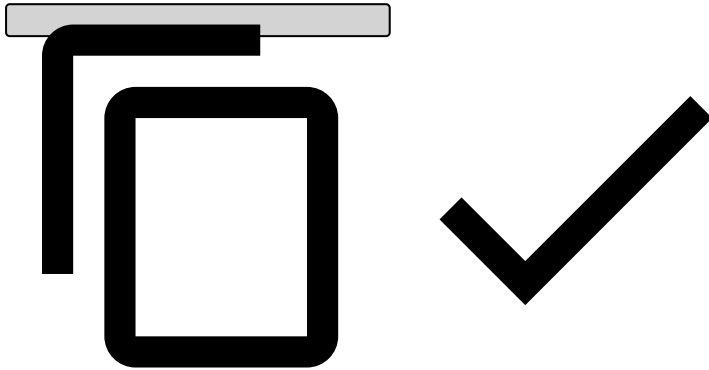
Make sure that you silence your patterns before exiting the server! If you report for a given pattern, shutting down the server and leaving interactive mode does not stop the pattern from being reported.

Since 15.1.11 metrics server will silence all probes when successfully shutdown.

Example

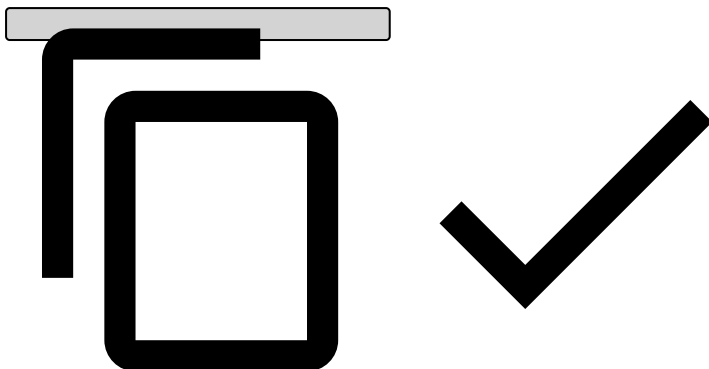
Start by running the interactive mode and list all the available patterns.

```
$ l
(...)
[email protected] not reporting, schema:
(uuid:STRING,timestamp:LONG,msg_type:STRING,rule_process_latency:LONG,rule_total_latency:LONG)
[email protected] not reporting, schema:
(uuid:STRING,timestamp:LONG,message_type:STRING,null_instance:BOOLEAN,scoring_fetchinstance_latency:LONG,scoring_fos_latency:LONG,scoring_total_latency:LONG)
[email protected] not reporting, schema:
(uuid:STRING,timestamp:LONG,msg_type:STRING,rule_process_latency:LONG,rule_total_latency:LONG)
(...)
```



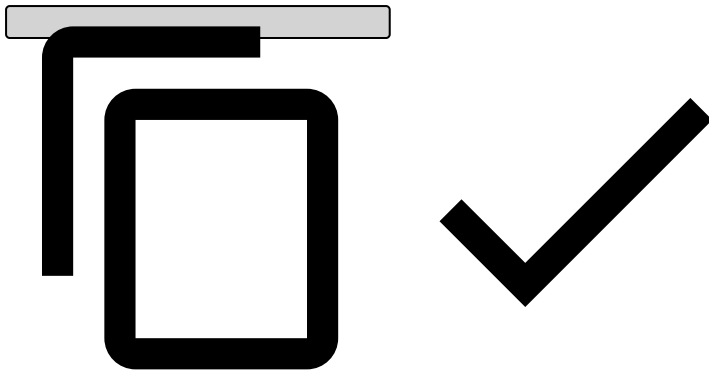
Select one of the available patterns, say `[email protected]` , to check its current status and schema.

```
$ r [email protected]
$ l [email protected]
[email protected] reporting, schema:
(uuid:STRING,timestamp:LONG,message_type:STRING,null_instance:BOOLEAN,scoring_fetchinstance_latency:LONG,scoring_fos_latency:LONG,scoring_total_latency:LONG)
```



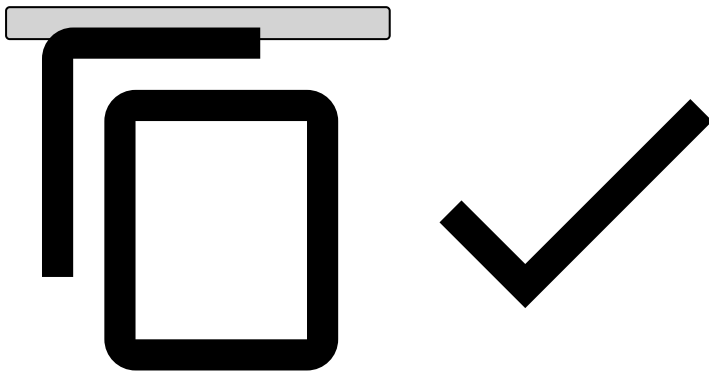
Enable reporting for the above pattern.

```
$ r [email protected]
$ l [email protected]
[email protected] reporting, schema:
(uuid:STRING,timestamp:LONG,message_type:STRING,null_instance:BOOLEAN,scoring_fetchinstance_latency:LONG,scoring_fos_latency:LONG,scoring_total_latency:LONG)
```



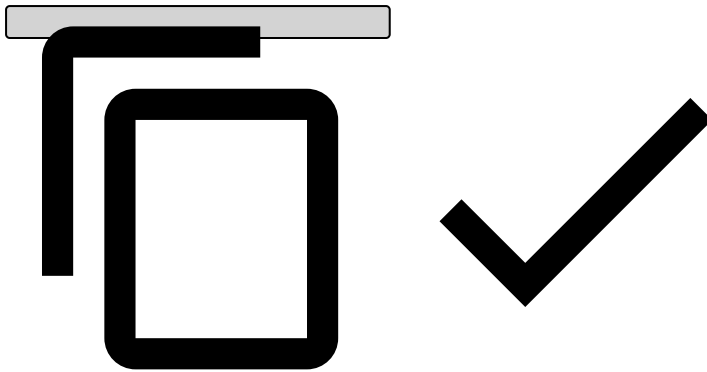
Once instances pass through the defined fraud scoring service, some lines will be generated in the output file, define in the configuration file. Note that the time is represented in nanoseconds.

```
mylocalhost,3538b328-134f-4d63-84cf-bd4c50db93e6,1465421548284,NORMAL,false,
1967209,377659,2426728
mylocalhost,65b02454-11b1-42e9-b2ba-f0438c5933e0,1465421549518,NORMAL,false,795394,272169,1116582
mylocalhost,8bbbed484-177e-4985-b835-7a4cb85c3950,1465421549676,NORMAL,false,844627,267752,1160902
mylocalhost,f62d3877-ed3b-4489-85fb-472c465a7c14,1465421549844,NORMAL,false,805862,239146,1107133
mylocalhost,2b892352-1c93-485b-9646-1ac18455a067,1465421550011,NORMAL,false,631836,219795,910742
mylocalhost,047e3b92-21b3-4703-8c97-5aa74765d9c5,1465421550158,NORMAL,false,903389,252477,1223005
mylocalhost,d5e2c84d-f1a8-46bb-a7ae-0157ce6433bf,1465421550481,NORMAL,false,289246,227748,558441
mylocalhost,edb06d6c-86fd-42bd-a2ed-1afe7bd742fa,1465421550683,NORMAL,false,796873,242708,1086698
mylocalhost,c5d1743f-6bd7-4e38-80c8-8f0220653d07,1465421550885,NORMAL,false,595244,233126,873616
mylocalhost,d3c5e203-b021-4100-af89-ece74af997b1,1465421551068,NORMAL,false,840138,238641,1124537
mylocalhost,811ad876-fdbd-410a-b15e-1dfe5ed363dc,1465421551277,NORMAL,false,747988,255783,1053942
mylocalhost,0f2de8ea-000e-4933-8307-c127e2bac71a,1465421551471,NORMAL,false,799524,253927,1102078
mylocalhost,40b65b62-c5ba-4e92-9124-a2f8143dda1a,1465421551640,NORMAL,false,678110,235491,988029
mylocalhost,7d1d57f7-0553-4434-afa3-ab29a726e6de,1465421552376,NORMAL,false,817818,258881,1125458
mylocalhost,69e3fa38-ae32-4840-a706-6a6afb031fc,1465421552705,NORMAL,false,814170,225829,1086237
mylocalhost,5caff4a5-2773-42f1-a1e2-d6b2df1d3637,1465421553119,NORMAL,false,783721,258148,1089784
mylocalhost,d22a1a12-fcc1-4187-a650-cb3f68d7bc64,1465421553350,NORMAL,false,864084,248861,1161982
mylocalhost,3eabdb18-8ff4-44a0-a20f-119464632d87,1465421553652,NORMAL,false,807886,240421,1100518
mylocalhost,7a14f56b-aa60-4900-86af-c4cb3519de65,1465421553863,NORMAL,false,663703,236307,946143
mylocalhost,30358947-0550-480a-aa57-8ab390465378,1465421554032,NORMAL,false,811868,238845,1096526
mylocalhost,f16f61e2-01c6-47d7-9668-503ea9742ac1,1465421554223,NORMAL,false,830213,242055,1133387
```



Before leaving the interactive mode, silence the reported pattern.

```
$ s [email protected]
$ l [email protected]
[email protected] not reporting, schema:
(uuid:STRING,timestamp:LONG,message_type:STRING,null_instance:BOOLEAN,scoring_fetchinstance_late
ncy:LONG,scoring_fos_latency:LONG,scoring_total_latency:LONG)
```



Metrics Server Properties

The `metrics.properties` has the following properties.

Property	Description	Default Value
<code>metrics.autostartprobes</code>	The server will register on the probes that match these patterns and that start after the metrics server starts. Patterns support the wildcard characters <code>*</code> and <code>?</code> . The <code>*</code> wildcard matches a sequence of zero or more characters and the <code>?</code> matches exactly one character.	None
<code>metrics.host</code>	The host of the metrics server. Probes will report connecting to this host.	hostname reported by <code>InetAddress.getLocalHost().getHostName()</code> or <code>localhost</code> if that fails.
<code>metrics.port</code>	The port the metrics server is listening on.	16161
<code>metrics.replenabled</code>	Whether the metrics server should run in interactive mode or not.	true
<code>metrics.store.outputdir</code>	Metrics will be stored to files in this folder. This folder will be created if does not exist.	Mandatory property.
<code>metrics.messaging.*</code>	Messaging can be configured with the usual messaging configuration properties prefixed by <code>metrics.messaging</code> . Must match the pulse-side messaging configuration.	Embedded ActiveMQ. Only works when probes and metrics server are on the same machine.

Existing Probes

The following probes are built-in in the Pulse Server/App Engines (where `<workflow_element>` stands for the service name sanitized):

Probe Target	Description	Probe Name	Schema	Notes
All Workflow Input Services	Latency associated to input event publishing. A metric record is generated for each event published into the workflow.	service_runner_input<workflow_element>	timestamp: Long	
			uuid: String	
			message_type: String	
			runner_total_latency: Long	- Services treating the message synchronously - Latency between the message entering the

				service's internal queue. The message enters the queue when <code>post()</code> is called on the message object. The message is then being ready to be processed by the next workflow element. • Services treating the message asynchronously Latency between the message entering the service's internal queue and the message being ready to enter the next workflow element. Note that in this case, the queuing of the message is ready to be consumed by the next element and not accounted for. The time the service spends treating the message and the time the message spends traversing to other workflow elements are included.
			<code>event_total_latency</code>: Long	The time it takes to process the full workflow and for a call-back to be sent to this input service runner informing that the event finished. This probe is the same as <code>runner_total_latency</code> with the additional time that it takes to completely treat the request throughout the workflow.
All Workflow Input Services	Latency associated with the time spent by a message in a workflow queue, waiting to be processed by the next workflow internal service.	<code>service_runner_input_queue_<workflow_element></code>	<code>timestamp</code>: Long <code>uuid</code>: String <code>message_type</code>: String <code>input_queue_latency</code>: Long	Å
	Latency associated with	<code>service_runner_<workflow_element></code>	<code>timestamp</code>: Long	Å

All Workflow Internal Services	the processing in an internal (i.e., non-input) service. This is measured at the "service runn level, i.e., the holder that encapsulates the service and allows it to run in the workflow (including custom services). A metric record is generated for each event that goes through such a runner.		uuid: String	Â
			message_type: String	Â
			runner_total_latency: Long	• Normal message handling: - Latency between the message entering the service's internal <code>post()</code> and the message being ready to enter the next workflow element. • Error treating the message - The latency between the message entering the internal service's <code>post()</code> and an error being caught. This includes the time to process the task on the service's <code>onMessage</code> and any setup before enqueueing the message to the workflow. The time taken to enqueue to the workflow for further processing is not included.
			failure: Boolean	Whether there was an error condition or some exception thrown while processing a <code>post()</code> message.
Workflow Rules Service	Latency for processing all the rules which were evaluated for an event. A metric record is generated for each event processed in each rules service.	rules_service_<workflow_element>	timestamp: Long	Â
			uuid: String postLatency: Long	
Workflow Scoring Service	Latency for scoring an event.	scoring_service_<workflow_element>	timestamp: Long uuid: String	Â

	A metric record is generated for each event processed in each scoring service.		scoreLatency: Long	
Metric Fetcher	Latency for fetching metrics for a MetricListOperator of a plan used in a Plan Workflow Service. A metric record is generated for each event that traverses such an operator in each plan. Note that each Metric Fetcher has its own ID (shown in the 3rd column). This is because there may exist multiple plans and/or multiple metric operators with scheduled metrics.	scheduled_metrics_fetcher	timestamp: Long message_id: String fetcher_id: Long fetch_success: Boolean fetch_size: Integer fetch_selector_build_latency: Long fetch_profile_store_latency: Long fetch_resolve_latency: Long fetch_total_latency: Long	The <i>fetch_profile_store_latency</i> measures the time between triggering the fetch of the metric and either its successful retrieval or its cancellation due to timeout. These things are done in separate threads. Due to the way that the scheduling happens in the Java Virtual Machine, there is no guarantee that threads execute immediately when scheduled. As such, the times shown in <i>fetch_profile_store_latency</i> and <i>fetch_total_latency</i> entries might be greater than the actual time it took to either fetch or timeout.
Real-time PKernel	Latency for processing real-time events in each PKernel. This is analogous to the Metric Fetcher, but for the real-time part of the metrics calculation (whereas the Metric Fetcher is for the fetch of the scheduled metrics). Note that each PKernel has its own ID, also analogously to the Metric Fetcher. Furthermore, it	pkernel_realtime_metrics	timestamp: Long pkernel_id: Long partition: Long event_id: String latency: Long	Â

	has a partition identifier since the same (conceptual) PKernel (in a given Metric operator of a given plan) may be partitioned (according to the various existing workflow partitions).			
Real-time checkpointable PKernel (this only available when PKernel with persistent state is enabled)	Latency for processing real-time events in each PKernel. This is analogous to the Metric Fetcher, but for the real-time part of the metrics calculation (whereas the Metric Fetcher is for the fetch of the scheduled metrics). Note that each PKernel has its own ID, also analogously to the Metric Fetcher. Furthermore, it has a partition identifier since the same (conceptual) PKernel (in a given Metric operator of a given plan) may be partitioned (according to the various existing workflow partitions).	checkpointable_pkernel_post	timestamp: Long operator_name: String partition: Long pkernel_id: String event_id: String latency: Long	This probe is similar to pkernel_realtime_metric but it provides a scheduler that reports in addition the name of the operator in the Plan and the PKernel id. This only available when PKernel with persistent state is enabled.
GC Pauses	Probe used in the metrics agent to monitor garbage collection pauses from the jvm gc log file.	gcPausesMetric property configured in the metrics agent.	timestamp: Long gc_pause_in_nanos: Long	Å
End of the workflow	The latency spent at the end of the workflow.	post_workflow	timestamp: Long uuid: String	Current timestamp in milliseconds. Id of the message.

			message_type: String	The type of the message.
			deadletters_notify_before_event_storage_latency: Long	The latency (in nanoseconds) for the time spent at the end of the workflow notifying deadletters listeners before storing the message in the Event Storage.
			deadletters_notify_after_event_storage_latency: Long	The latency (in nanoseconds) for the time spent notifying deadletters listeners after storing the message in the Event Storage.
			eventstorage_queue_latency: Long	The latency (in nanoseconds) for the time spent at the end of the workflow sending the message to the pending queue which will then asynchronously save it to the Event Storage.
			processing_total_latency: Long	The latency (in nanoseconds) for the time spent at the end of the workflow.
workflow_type: Long	The value is 1 if the message was from the past, or 0 if the message was a real-time one.			
DynamoDB BatchGetItem	Latency spent by metric fetcher on each BatchGetItem operation.	batch_get_item	timestamp: Long	Current timestamp in milliseconds.
			message_id: String	Id of the message.
			fetcher_id: Long	The fetcher id.
			batch_num: Long	The BatchGetItem being performed.
			batch_total: Long	The total number of BatchGetItem requests to be performed.
			unprocessed: Boolean	If the BatchGetItem has unprocessed items that require a retry.
			latency: Long	The latency of the BatchGetItem.

Probe names include the pulse node ip address appended to what is specified in the "Probe Name" column.

Storing Metric Data Locally

Probes can also be configured to write metric data directly to a local text file, using the logging framework. This setup doesn't require a metric server but allows only for metric data to be stored locally on (each) Pulse Server and doesn't allow individual probes to be "disabled", as when using the metric server.

The Pulse standard log configuration file (`PULSE_HOME/conf/logger/logger-metrics.xml`) includes a commented out section with an appender (`METRICS-ASYNC` , that in turn uses the `METRICS` appender) and a logger for that appender. In order to have the probes writing directly to files it is enough to uncomment this section.

```
<!--
  Start of metrics server local logging.
  By default this logger is disabled. This means that probes will not be logged to disk,
  only reported to the metrics server if enabled.

  With this logger metrics server doesn't need to be enabled.

  Use an AsyncAppender to avoid impacting the critical path of execution degrading processing
  latencies.
-->
<!-- METRICS SERVER LOGGING TAG
<appender name="METRICS" class="ch.qos.logback.core.rolling.RollingFileAppender">
  <file>log/${pulse.logcontext}-metrics.log</file>
  <encoder>
    <pattern>%msg</pattern>
  </encoder>
  <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
    <fileNamePattern>log/${pulse.logcontext}-metrics.%d{yyyy-MM}-week-%d{WW}.log.gz</
fileNamePattern>
    <maxHistory>8</maxHistory>
  </rollingPolicy>
</appender>

<appender name="METRICS-ASYNC" class="ch.qos.logback.classic.AsyncAppender">
  <appender-ref ref="METRICS" />
  <queueSize>2048</queueSize>
  <discardingThreshold>0</discardingThreshold>
  <includeCallerData>false</includeCallerData>
</appender>

<logger name="com.feedzai.metrics.probes.TextAbstractProbe" level="TRACE" additivity="false">
  <appender-ref ref="METRICS-ASYNC" />
</logger>
METRICS SERVER LOGGING TAG -->
<!-- End of metrics server local logging. -->
```

